

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 10

Recap and outline

- Wrapper methods
- Re-training
- AIC and BIC
- Forward selection

Wrapper methods

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1, 2, \dots, p\}, |S| \leq s} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

- Given a class of prediction models/functions f , for each $S \subset \{1, 2, \dots, p\}$ we solve find the size of the objective
- Applies to any function class $\mathcal{F}$ (later we focus on a neural networks approach)
- Huge computational challenge, need to evaluate every loss function $\mathcal{L}(., .)$ for 2^p or $\binom{p}{s}$ subsets

Optimization challenges

- Traverse the space of $\binom{p}{s}$ or 2^p in clever ways
- Evaluate each feature individually (LOCO (leave-one-covariate-out))
- Start with an empty set and *greedily* add features (forward selection)
- Start with a full subset and remove features (feature elimination/backward selection)

$$\mathbb{E}[Y|X = x] = f_{\theta}(x)$$

where θ is a set of parameters defining prediction function

- Examples (θ could be regression co-efficients, weights in neural networks, e.t.c.)
- When feature subset changes, θ also re-trained in order to optimize loss/performance
- Need to learn a different prediction function/parameter set for each subset S

AIC and BIC

- Classical statistical feature selection methods (1970's)
- Similar criteria that come from very different perspectives
- Computationally combinatorial (NP-hard) problem

AIC (Akaike Information Criterion)

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}} \min_{\beta_S} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, \sum_{j \in S} \beta_j X_j^{(i)}) + |S|$$

BIC (Bayesian Information Criterion)

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}} \min_{\beta_S} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, \sum_{j \in S} \beta_j X_j^{(i)}) + \frac{|S| \log n}{2}$$

Principle behind AIC

- Based on a criterion for minimizing expected out of sample prediction error
- Penalize non-zero co-efficients to help avoid over-fitting on out-of-sample data
- *Like* choosing feature set based on cross-validated prediction error

$$\begin{aligned}\text{AIC critereion} &= \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, \sum_{j \in S} \beta_j X_j^{(i)}) + |S| \\ &\approx \mathbb{E}_{(X^{new}, Y^{new})} [\mathcal{L}(Y^{new}, \sum_{j \in S} \beta_j X_j^{new})]\end{aligned}$$

Principle behind BIC

- Based on a criterion for *Bayesian model selection*
- Start from a belief that parameter 0 and then only includes if statistically significant
- *Like* choosing feature set based on p-values under a model

$$\begin{aligned}\text{BIC critereion} &= \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, \sum_{j \in S} \beta_j X_j^{(i)}) + \frac{|S| \log n}{2} \\ &\approx \int \mathcal{L}(Y, \sum_{j \in S} \beta_j X_j) \pi(\beta_S) d\beta\end{aligned}$$

where $\pi(\beta_S)$ is a prior belief distribution.

The trade-off between prediction and feature selection

- Both AIC and BIC penalize models and induce sparsity
- BIC has a harsher penalty than AIC
- The goal of AIC is to select features that improve out-of-sample prediction error
- The goal of BIC is to select only features that are statistically significant
- Quite deep and subtle and maybe counter-intuitive that these are different
- Features that have a weak effect may be selected by AIC but not BIC

Why is it a wrapper method?

AIC (Akaike Information Criterion)

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}} \min_{\beta_S} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, \sum_{j \in S} \beta_j X_j^{(i)}) + |S|$$

BIC (Bayesian Information Criterion)

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}} \min_{\beta_S} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, \sum_{j \in S} \beta_j X_j^{(i)}) + \frac{|S| \log n}{2}$$

- Solving an optimization problem (like Lasso) so why is it a wrapper
- All down to the fact that we optimize over the feature subset outside the prediction (need to search over all subsets)

Forward selection with AIC and BIC

- Start with an empty set and add one feature at a time
- Select feature that has the greatest improvement of the objective (need to try out p times)
- Continue iteratively
- Stop when criteria does not improve
- Applies to any loss function/objective (e.g. neural networks, ensemble trees e.t.c.), we simply use AIC/BIC here
- **Challenges:**
 - ▶ Highly dependent features
 - ▶ Non-linear dependence
 - ▶ Lots of features with weak signals

Forward selection code

```
: def forward_selection(X, y, criterion="aic"):
    remaining = list(X.columns)
    selected = []
    scores = []

    while remaining:
        results = []

        for candidate in remaining:
            features = selected + [candidate]
            X_model = sm.add_constant(X[features])
            model = sm.OLS(y, X_model).fit()

            score = model.aic if criterion == "aic" else model.bic
            results.append((score, candidate))

        results.sort()
        best_score, best_candidate = results[0]

        selected.append(best_candidate)
        remaining.remove(best_candidate)
        scores.append(best_score)

    return selected, scores

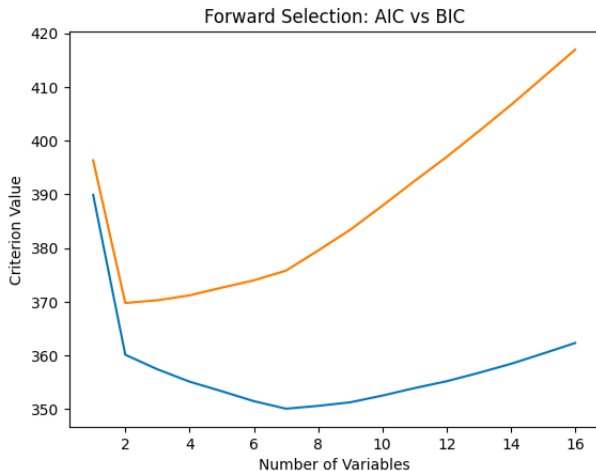
aic_vars, aic_scores = forward_selection(X_train, y_train, "aic")
bic_vars, bic_scores = forward_selection(X_train, y_train, "bic")
```

Backward selection and taking backward steps

- You can apply exactly the same principle where we start with full model and remove features
- For example LOCO or recursive feature elimination for tree ensembles
- Some algorithms take both forward and backward steps

AIC vs BIC criteria

```
: plt.figure()
plt.plot(range(1, len(aic_scores)+1), aic_scores)
plt.plot(range(1, len(bic_scores)+1), bic_scores)
plt.xlabel("Number of Variables")
plt.ylabel("Criterion Value")
plt.title("Forward Selection: AIC vs BIC")
plt.show()
```



Recall: Lasso

Lasso optimization problem:

$$\widehat{\beta} \in \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, \sum_{j=1}^p \beta_j X_j^{(i)}) + \lambda \sum_{j=1}^p |\beta_j|$$

Recall that selecting λ is a critical decision that influences which and how many features are selected

Lasso path code

```
X_train_const = sm.add_constant(X_train)
ols_model = sm.OLS(y_train, X_train_const).fit()
```

```
lasso_cv = LassoCV(cv=5, random_state=0)
lasso_cv.fit(X_train_scaled, y_train)
```

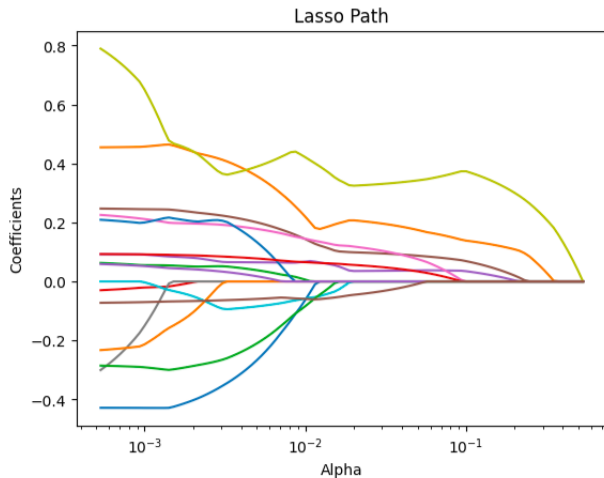
```
lasso_coef = pd.Series(lasso_cv.coef_, index=X.columns)
```

```
alphas, coefs, _ = lasso_path(X_train_scaled, y_train)
```

```
plt.figure()
for coef in coefs:
    plt.plot(alphas, coef)
```

```
plt.xscale("log")
plt.xlabel("Alpha")
plt.ylabel("Coefficients")
plt.title("Lasso Path")
plt.show()
```

Lasso path



Motivating example: Hitters baseball data

- Predicting salary for Baseball players on 1987 (from Kaggle)
- $n = 322$, $p \approx 20$
- 3 "types" of features
 - ▶ Performance stats from previous season (1986): home runs, hits, runs, e.t.c.
 - ▶ Performance stats over career
 - ▶ Other: Years, errors, e.t.c

Hitters data pre-processing

```
df = pd.read_csv("Hitters.csv")

df = df.dropna()

# Response
y = np.log(df["Salary"]) # log-transform (standard in ISLR)

# Predictors (drop Salary + categorical League)
X = df.drop(columns=["Salary", "League", "Division", "NewLeague"])

# One-hot encode categoricals if desired (optional)
# X = pd.get_dummies(X, drop_first=True)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

# Standardize for Lasso
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

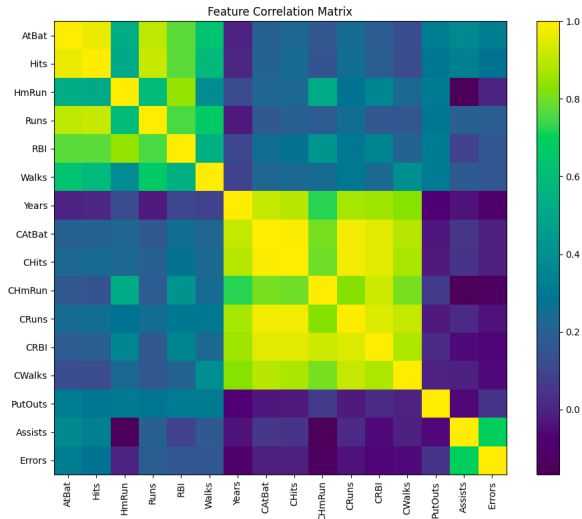
Hitters features correlation matrix code

```
plt.figure(figsize=(10,8))

plt.imshow(corr_matrix)
plt.colorbar()
plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns, rotation=90)
plt.yticks(range(len(corr_matrix.columns)), corr_matrix.columns)

plt.title("Feature Correlation Matrix")
plt.tight_layout()
plt.show()
```

Hitters features correlation matrix



OLS code for Hitters

```
ols_model = sm.OLS(y_train, sm.add_constant(X_train)).fit()
```

```
summary_df = pd.DataFrame({
    "Coefficient": ols_model.params,
    "Std_Error": ols_model.bse,
    "t_value": ols_model.tvalues,
    "p_value": ols_model.pvalues
})

# Drop intercept
summary_df = summary_df.drop("const")

# Add significance flag (alpha = 0.05)
alpha = 0.05
summary_df["Significant_0.05"] = summary_df["p_value"] < alpha

# Sort by p-value
summary_df = summary_df.sort_values("p_value")

print(summary_df)
```

Comparing 4 methods

```
best_aic_idx = np.argmin(aic_scores)
best_bic_idx = np.argmin(bic_scores)

best_aic_vars = aic_vars[:best_aic_idx + 1]
best_bic_vars = bic_vars[:best_bic_idx + 1]

# Fit final models
model_aic = sm.OLS(
    y_train, sm.add_constant(X_train[best_aic_vars])
).fit()

model_bic = sm.OLS(
    y_train, sm.add_constant(X_train[best_bic_vars])
).fit()
```

```
all_vars = X.columns

coef_df = pd.DataFrame(index=all_vars)

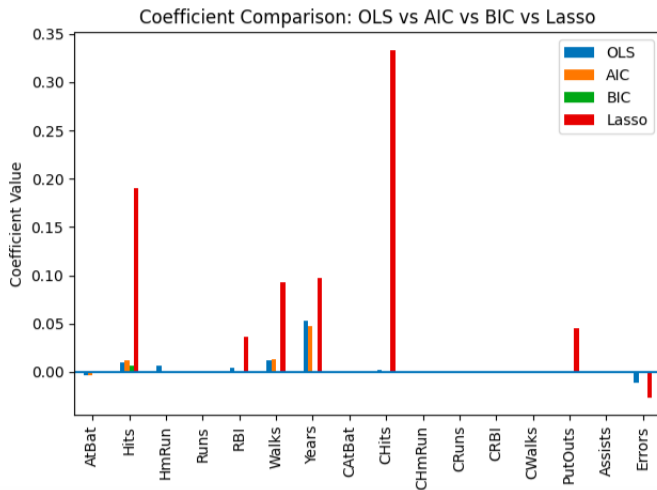
# OLS
coef_df["OLS"] = ols_model.params.reindex(all_vars).fillna(0)

# AIC
coef_df["AIC"] = model_aic.params.reindex(all_vars).fillna(0)

# BIC
coef_df["BIC"] = model_bic.params.reindex(all_vars).fillna(0)

# Lasso
coef_df["Lasso"] = lasso_coef.reindex(all_vars).fillna(0)
```

Methods comparison



Co-efficients for Hitters

coef_df["OLS"]

AtBat	-0.002922
Hits	0.009666
HmRun	0.006575
Runs	-0.001136
RBI	0.004379
Walks	0.011912
Years	0.052839
CAtBat	-0.000239
CHits	0.001848
CHmRun	0.000896
CRuns	0.000524
CRBI	-0.001217
CWalks	-0.001123
PutOuts	0.000330
Assists	0.000458
Errors	-0.011013

coef_df["Lasso"]

AtBat	0.000000
Hits	0.190776
HmRun	0.000000
Runs	0.000000
RBI	0.036402
Walks	0.093055
Years	0.096882
CAtBat	0.000000
CHits	0.333652
CHmRun	-0.000000
CRuns	0.000000
CRBI	0.000000
CWalks	0.000000
PutOuts	0.044891
Assists	-0.000000
Errors	-0.026591

coef_df["AIC"]

AtBat	-0.003090
Hits	0.012432
HmRun	0.000000
Runs	0.000000
RBI	0.000000
Walks	0.012704
Years	0.047669
CAtBat	0.000000
CHits	0.000858
CHmRun	0.000000
CRuns	0.000000
CRBI	0.000000
CWalks	-0.001230
PutOuts	0.000308
Assists	0.000000
Errors	0.000000

: coef_df["BIC"]

AtBat	0.000000
Hits	0.006255
HmRun	0.000000
Runs	0.000000
RBI	0.000000
Walks	0.000000
Years	0.000000
CAtBat	0.000000
CHits	0.000793
CHmRun	0.000000
CRuns	0.000000
CRBI	0.000000
CWalks	0.000000
PutOuts	0.000000
Assists	0.000000
Errors	0.000000

Discussion of selected features

- BIC tends to recover a sparser subset (since optimizing for model selection)
- Lasso and AIC have quite similar results (since both are optimizing for prediction performance)
- Due to high correlation within 3 blocks, many of the features interchangeable
- BIC and OLS after thresholding based on p-values have similar results

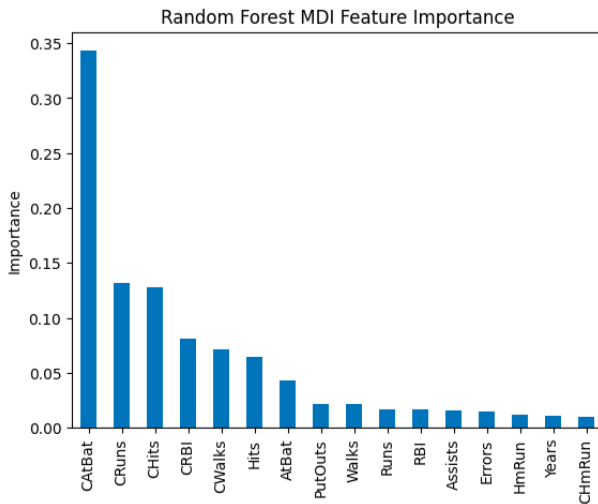
MDI feature importance for Hitters code

```
: rf = RandomForestRegressor(random_state=0)
  rf.fit(X_train, y_train)

  mdi_importance = pd.Series(rf.feature_importances_, index=X.columns)
  mdi_importance = mdi_importance.sort_values(ascending=False)

: plt.figure()
  mdi_importance.plot(kind="bar")
  plt.title("Random Forest MDI Feature Importance")
  plt.ylabel("Importance")
  plt.show()
```

MDI feature importance



Prediction comparison of all 5 methods

```
# AIC model
X_test_aic = sm.add_constant(X_test[aic_vars])
model_aic = sm.OLS(y_train, sm.add_constant(X_train[aic_vars])).fit()
pred_aic = model_aic.predict(X_test_aic)

# BIC model
X_test_bic = sm.add_constant(X_test[bic_vars])
model_bic = sm.OLS(y_train, sm.add_constant(X_train[bic_vars])).fit()
pred_bic = model_bic.predict(X_test_bic)

# OLS
pred_ols = ols_model.predict(sm.add_constant(X_test))

# Lasso
pred_lasso = lasso_cv.predict(X_test_scaled)

# RF
pred_rf = rf.predict(X_test)

results = {
    "AIC": mean_squared_error(y_test, pred_aic),
    "BIC": mean_squared_error(y_test, pred_bic),
    "OLS": mean_squared_error(y_test, pred_ols),
    "Lasso": mean_squared_error(y_test, pred_lasso),
    "RF (MDI)": mean_squared_error(y_test, pred_rf)
}

print(results)
```

```
{'AIC': 0.44766051049000394, 'BIC': 0.44766051049000394, 'OLS': 0.44766051049000427, 'Lasso': 0.39299242437475396, 'RF (MDI)': 0.1439043833468212}
```

Discussion of all 5 method results

- MDI clearly has best prediction performance (includes all features and very different model)
- Lasso also has better prediction performance than OLS, AIC, BIC
- Again illustrates tradeoff between sparsity and prediction error performance

Question: How do we compare and determine a single set of selected features?